

レジスタ割り当て入門 — スタックマシンを卒業する

今日のゴール

式の途中結果の退避先を、メモリから **t** レジスタに変える。**fixed15** を全通させたまま、生成命令数をさらに減らす。

1 この回の位置づけ

- バックエンド発展シリーズの第 2 回（選択制）。前提はコマ 8（関数呼び出し）まで
- B1 と同じく **mycc.py** は書き換えない。ラッパー **regcc.py**（完成品）が、退避まわりのメソッドだけを差し替える
- 編集するのは **regstack.py** の 4 つの関数だけ
- B1 とは独立している（どちらから取り組んでもよい）

2 いまのコンパイラは「スタックマシン」

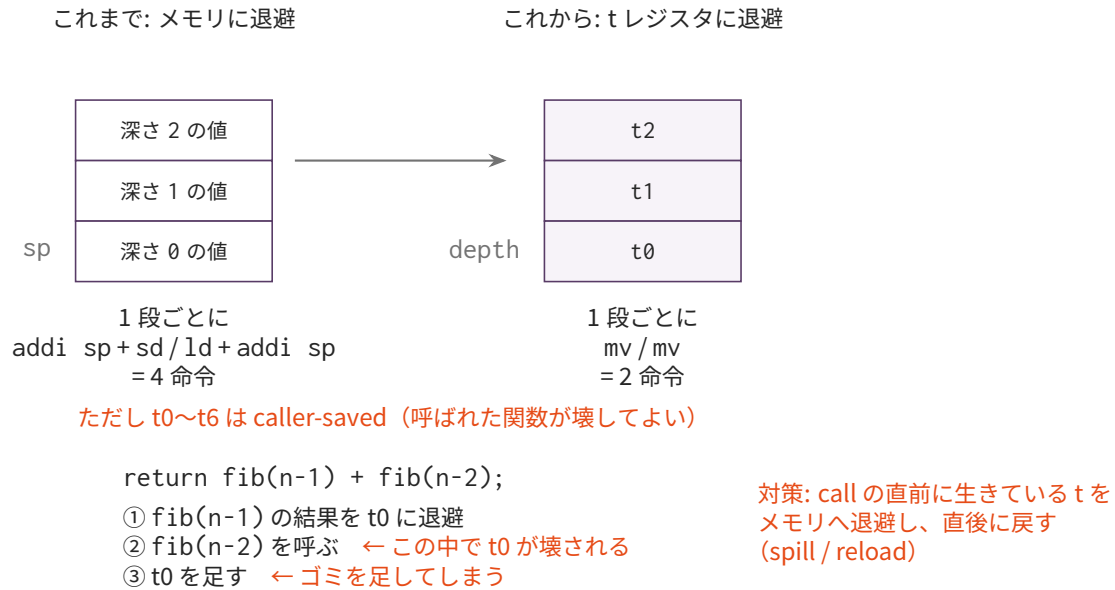
コマ 3 以来、二項演算はこう生成されてきた。

```
# 左辺を計算して a0 へ
addi sp, sp, -8
sd a0, 0(sp)      ← メモリへ退避
# 右辺を計算して a0 へ
ld a1, 0(sp)      ← メモリから復元
addi sp, sp, 8
add a0, a1, a0
```

この方式は単純で常に正しいという大きな長所がある。式がどれだけ深くネストしても、スタックが深くなるだけで破綻しない。だからこそ講義の最初に採用した。

しかし、退避 1 段につき 4 命令、しかもメモリアクセスが 2 回入る。レジスタは 32 本もあるのに、ほとんど使っていない。

t0～**t6** を「深さ 0 の値は **t0**、深さ 1 の値は **t1**、…」と割り当てれば、退避は **mv 1** 命令、復



深さが 7 段を超えた分だけメモリを使う。浅い式 (ほとんどの式) はレジスタだけで済む

図 1 メモリ退避とレジスタ退避、そして call の問題

元も mv 1 命令で済む。4 命令が 2 命令になる。

3 depth — 何段目かはコンパイル時に分かる

鍵になるのは、push と pop が必ず対になっていることである。codegen は式を再帰的にたどるので、退避の深さはコンパイル時に静的に決まる。

```
(a + b) * (c + d)
```

深さ 0: (a+b) の結果を退避 → t0

深さ 1: c の結果を退避 → t1 ← (c+d) を計算中

深さ 1: t1 を復元して c+d

深さ 0: t0 を復元して掛け算

そこで、コード生成器に depth(いま何段退避しているか) を持たせ、push_a0 で 1 増やし、pop_into で 1 減らす。depth 段目の値は REGS[depth] に置く、と決めればよい。

レジスタが尽きたら (8 段以上ネストしたら)、これまで通りメモリへ退避する。浅い式はレジスタ、深い式はメモリという使い分けである。実際のプログラムでは、式が 8 段もネストすることはめったにない。

4 落とし穴: t レジスタは call で壊れる

RV64 の呼び出し規約では、`t0`～`t6` は **caller-saved** (一時レジスタ) である。つまり「呼ばれた関数は自由に壊してよい。必要なら呼ぶ側が守れ」という約束になっている。

```
return fib(n - 1) + fib(n - 2);
```

`fib(n-1)` の結果を `t0` に退避してから `fib(n-2)` を呼ぶと、その中の `fib` が同じ `t0` を退避に使うので、値が壊れる。コマ 8 の資料で「**callee-saved / caller-saved**」を「この回では基本だけ扱う」と先送りにした話が、ここで効いてくる。

対策は素直に、**call** の直前に生きている **t レジスタ** をメモリへ退避し、直後に戻す (spill / reload)。呼び出しがない式ではこの退避は発生しないので、ほとんどの式は `mv` だけで済む。

補足

なぜ **s レジスタ (callee-saved)** を使わないのか。 `s1`～`s11` を使えば呼び出しをまたいでも壊れない。ただしその場合は、自分の関数のプロローグで **s レジスタ** を保存し、エピローグで戻す義務が生じる (自分も「呼ばれた関数」だから)。フレームサイズの計算も変わる。「呼び出しの周りだけ退避する」方が、コマ 7 で作ったフレーム生成に手を入れずに済む。どちらが得かはプログラム次第で、本物のコンパイラは「呼び出しをまたぐ値は **s**、またがない値は **t**」と使い分けている。

5 実装

Step	実装対象	内容
1	<code>push_a0 / pop_into</code>	<code>depth</code> に応じてレジスタ / メモリ
2	<code>spill_before_call / reload_after_call</code>	<code>call</code> をまたぐ <code>t</code> の退避

関数は `cg` (コード生成器のインスタンス) を受け取り、`cg.emit(行)` で命令を出し、`cg.depth` を読み書きする。

5.1 sp の整列に注意

メモリ退避に落ちたときは、**8 バイトではなく 16 バイト単位**で `sp` を動かす。8 バイト単位だと、退避が奇数段のときに `sp` が 16 バイト境界からずれ、その状態で `call` すると ABI 違反になる（コマ 7 のアラインメント問題の再来）。`spill` も同じ理由で、16 の倍数に切り上げて確保する。

5.2 動かし方

```
python3 regcc.py ../../../../final/tests/f09_recur.c
python3 regcc.py ../../../../final/tests/f09_recur.c | python3 ../count_insns.py
```

`regcc.py` は `mycc.py` の `_push_a0` / `_pop_into` / `_gen_call` を自分の `regstack.py` の実装に差し替えて実行する（`_gen_call` は引数の受け渡しに `push/pop` を使うので、あわせて差し替えている）。

6 テスト

```
python3 check.py      # Step ごとの単体テスト（未実装は SKIP）
python3 golden.py     # fixed15 全通 + 命令数の before/after
```

`check.py` は `emit` と `depth` だけを持つ「コード生成器もどき」に対して関数を呼び、出力される行を直接確かめる。

全テスト **PASS** かつ命令数が減っていることが完了条件である。教員の実装では合計 1018 → 812 命令（約 20% 減）になる。

重要

テストが落ちたら、まず次の 3 点を疑う。

1. `push_a0` と `pop_into` で `depth` の増減が対称か（`depth` を減らす前に読んでいないか）
2. `call` をまたぐ `t` レジスタを退避しているか（再帰のテストで露見する）
3. メモリ退避で `sp` の増減が一致しているか（-16 したなら +16 で戻す）

7 発展課題

1. **B1 と組み合わせる:** ピープホールの「push/li/pop 圧縮」規則は、レジスタ版では別のパターン (`mv t0, a0 + li a0, C`) になっている。新しい規則を書いて、さらに減らせるか試す
2. **使うレジスタを減らしてみる:** REGS を `['t0']` だけにすると何が起きるか。命令数とテスト結果を確認し、レジスタ数と性能の関係を体感する
3. **s レジスタ版:** 呼び出しをまたぐ値だけ `s1 ~ s11` に置く方式を設計する。プロローグ/エピローグで何を保存する必要があるか
4. **動的命令数を測る:** `qemu-riscv64 -d in_asm` など実際に実行された命令数を数え、静的な命令数の削減と比べる

補足

コラム: これは「レジスタ割り当て」なのか。本物のレジスタ割り当ては、変数の生存区間 (live range) を求め、干渉グラフを彩色して「どの変数をどのレジスタに置くか」を決める、コンパイラ最適化の花形である。この回でやったのは、式の途中結果に深さの順で機械的にレジスタを割り当てる簡易版で、ローカル変数はスタックに置いたままである。それでも 2 割減る。次の一步は「ループ内で何度も読む変数をレジスタに載せる」ことで、そこからが本当の割り当て問題になる。